University of Tripoli
Computer Engineering Department
Intro. To Computer Systems – EC351
Project Report
Spring 2025



Piano Sound Generation
Musical Notes via RISC-V Assembly

Student Names:

Ahmed Hawas 2210244424

Moayed Eshbeni 2210249584

Omar Azouze 2220209280

Due Date:1/7/2025

1.Objective:

The objective of this RISC-V assembly project is to simulate a digital piano interface that allows the user to generate musical notes through keyboard input using the RARS simulator's MIDI sound capabilities. The program maps specific keys to corresponding MIDI note values across two octaves, enabling real-time audio playback of individual notes. Additionally, it includes functionality to automatically play a predefined melody using synchronous MIDI note playback, with an enforced time limit.

2.Theory:

This RISC-V assembly project demonstrates how to generate musical notes through MIDI using the RARS simulator. The program maps keyboard keys to corresponding MIDI note numbers stored in a lookup table. When a user presses a key, the program identifies the character, determines the correct octave, and plays the note using an asynchronous MIDI syscall.

Additionally, the program includes a predefined melody composed of MIDI note values and corresponding durations, which is played using a synchronous syscall. The playback is limited to 5 seconds for controlled execution. The project showcases key assembly concepts such as memory access, syscall usage, pointer arithmetic, character processing, and control flow using loops and branches. It serves as a practical example of how low-level programming can interact with digital music protocols like MIDI in a simulated environment.

3.Design:

The program is designed around two main functionalities: real-time piano key input playback and automatic song playback.

1.Data Structures:

- note_table holds MIDI note numbers for two octaves (lower and higher).
- key_mapping stores the keyboard characters mapped to notes.
- The_Song and note_duration arrays store the melody notes and their durations.

2.User Input Handling:

The main loop waits for keyboard input via a syscall. It checks for special commands:

- Q/q to quit.
- S/s to play the predefined melody.
- Other keys are mapped to notes based on key_mapping.

3.Key Processing and Octave Detection:
Input characters are converted to uppercase to simplify matching. The program compares the original input with the uppercase version to detect whether Shift (uppercase) was pressed, deciding the octave accordingly.

4.Note Playback:
For individual key presses, the program retrieves the MIDI note from note_table and plays it asynchronously using a syscall, allowing immediate response to further input.

5.Melody Playback:
On command, the program plays The_Song synchronously, iterating through each note and its duration, with a total time limit of 5 seconds to prevent indefinite playback.

6.Control Flow:
Branch instructions handle loops, input validation, and program exit conditions, ensuring smooth interaction and response.

This modular design allows the program to efficiently manage input, note mapping, and sound output in a low-level environment.

Syscalls Used in the Code:

a7 Value	Syscall Name	Purpose
4	print_string	Prints a string to the console
10	exit	Terminates the program
12	read_char	Reads a character from keyboard input (no echo)
30	set_midi_tempo	Sets the tempo of MIDI playback
31	play_note_async	Plays a MIDI note (non-blocking)
33	play_note_sync	Plays a MIDI note (blocking/synchronous)

Summary of All MIDI Notes Used:

MIDI note numbers and center frequencies:

MIDI note number	Key number (Organ)	Key number (Piano)	Note names (English)	Frequency (Equal tuning at 440 Hz)
60	25	40	C4	261
62	27	42	D4	293
64	29	44	E4	329
65	30	45	F4	349
67	32	47	G4	392
69	34	49	A4	440
71	36	51	B4	493
72	37	52	C5	523
74	39	54	D5	587
76	41	56	E5	659
77	42	57	F5	698
79	44	59	G5	783
81	45	61	A5	880
83	45	63	B5	987

4.Implementation:

The program is implemented in RISC-V assembly and executed using the RARS simulator, which supports MIDI sound playback via system calls. The implementation is divided into the following stages:

1.Initialization:
The program sets the MIDI tempo using syscall 30, then prints a user prompt with available options using syscall 4.

2.Input Handling Loop:
A loop waits for keyboard input using syscall 12. If the user presses Q or q, the program exits. If S or s is pressed, it jumps to the melody playback routine. Otherwise, the program checks whether the input matches a valid musical key.

3.Key Mapping and Octave Selection:
The input character is compared against the key_mapping string after converting to uppercase. If the original input was uppercase (Shift held), the program selects notes from the higher octave by adding an offset. The final index is used to load the appropriate MIDI note from note_table.

4.Note Playback:
Once the MIDI note is determined, the program plays the note asynchronously using syscall 31, allowing the program to continue without waiting for the note to finish.

5.Melody Playback (The_Song):
When the user chooses to play the melody, the program loads note values and durations from The_Song and note_duration. It uses syscall 33 to synchronously play each note one at a time. A time counter ensures that playback stops after 5 seconds.

6.Memory and Registers:
Memory is used for storing note values, durations, and strings. Registers (t0–t6, a0–a3, a7) manage control logic, data access, and syscall parameters. This implementation showcases real-time MIDI interaction, low-level I/O handling, and structured control flow within a minimal RISC-V assembly environment.

- Main Code:

```
.data
note_table:
    .word 60
    .word 62
    .word 64
    .word 65
    .word 67
    .word 69
    .word 71
    .word 72
    .word 74
    .word 76
    .word 77
    .word 79
    .word 81
    .word 83
key_mapping: .asciz "ZXCVBNM"
The_Song:
    .word 60, 60, 67, 67, 69, 69, 67
    .word 65, 65, 64, 64, 62, 62, 60
    .word 67, 67, 65, 65, 64, 64, 62
    .word 67, 67, 65, 65, 64, 64, 62
    .word 60, 60, 67, 67, 69, 69, 67
    .word 65, 65, 64, 64, 62, 62, 60
Song_length: .word 42
note_duration: .word 500, 500, 500, 500, 500, 500, 1000
               .word 500, 500, 500, 500, 500, 500, 1000
               .word 500, 500, 500, 500, 500, 500, 1000
               .word 500, 500, 500, 500, 500, 500, 1000
               .word 500, 500, 500, 500, 500, 500, 1000
               .word 500, 500, 500, 500, 500, 500, 1000
prompt: .asciz "Press:\n- ZXCVBNM for lower octave\n- SHIFT+ZXCVBNM for higher octave\n- S to play The Song\n- Q to quit\n"
newline: .asciz "\n"
playing_msg: .asciz "Playing The Song...\n"
stop_msg: .asciz "Stopping playback after 5 seconds...\n"
.text
.globl main
main:
    li a7, 30
    li a0, 500000
    ecall
    la a0, prompt
    li a7, 4
    ecall
input_loop:
    li a7, 12
    ecall
    li t0, 'Q'
    beq a0, t0, exit
    li t0, 'q'
    beq a0, t0, exit
    li t0, 'S'
    beq a0, t0, play_The_Song
    li t0, 's'
    beq a0, t0, play_The_Song
    mv t6, a0
    li t1, 'a'
    blt a0, t1, is_uppercase
    addi a0, a0, -32
is_uppercase:
    la t3, key_mapping
    li t4, 0
search_loop:
    lb t5, 0(t3)
    beqz t5, invalid_note
    beq a0, t5, found_note
    addi t3, t3, 1
    addi t4, t4, 1
    j search_loop
found_note:
    beq t6, a0, shifted_key
    j unshifted_key
shifted_key:
    addi t4, t4, 7
unshifted_key:
    li t5, 14
    bge t4, t5, invalid_note
    slli t4, t4, 2
    la t5, note_table
    add t5, t5, t4
    lw a0, 0(t5)
    li a7, 31
    li a1, 300
    li a2, 0
    li a3, 127
    ecall
    j input_loop
play_The_Song:
    la a0, playing_msg
    li a7, 4
    ecall
    la t0, The_Song
    la t1, note_duration
    lw t2, Song_length
    li t3, 0
    li t4, 0
    li t5, 5000
Song_loop:
    bge t3, t2, input_loop
    bge t4, t5, stop_playback
    lw a0, 0(t0)
    lw a1, 0(t1)
    add t4, t4, a1
    li a7, 33
    li a2, 0
    li a3, 127
    ecall
    addi t0, t0, 4
    addi t1, t1, 4
    addi t3, t3, 1
    j Song_loop
stop_playback:
    la a0, stop_msg
    li a7, 4
    ecall
    j input_loop
invalid_note:
    j input_loop
exit:
    li a7, 10
    ecall
```

5.Debugging-Test-run:

The program was tested using the RARS simulator with MIDI support. Key inputs z–m and Z–M correctly played the corresponding notes from two octaves. Pressing S or s successfully played the predefined melody for up to 5 seconds. Input handling was verified to ignore invalid keys and exit properly on Q/q.

Common bugs included incorrect note mapping and melody overflow, which were fixed by verifying index bounds and ensuring proper shift detection. Debugging was done using RARS step-by-step execution and console prints. All features worked as expected.

6.Results analysis:

The program successfully achieved its objectives by allowing real-time musical note playback using keyboard input and MIDI system calls. Pressing keys Z–M and Shift+Z–Shift+M correctly triggered MIDI notes across two octaves, with accurate pitch mapping confirmed through audible output. The predefined melody (The_Song) played as intended, with proper timing and a 5-second playback limit enforced.

User input was responsive and robust, ignoring invalid keys without crashing. The use of asynchronous (syscall 31) and synchronous (syscall 33) MIDI calls allowed for real-time interaction and controlled playback. Register operations, memory access, and branching logic performed as expected, validating the correctness of the implementation.

Overall, the results demonstrate effective handling of MIDI playback, user input, and control flow in a low-level RISC-V assembly environment.

7.Conclusion and Future Improvements:

Conclusion:
This project successfully demonstrates how RISC-V assembly can be used to simulate a basic digital piano using MIDI note playback in the RARS simulator. It allows interactive key-based sound generation and melody playback through efficient use of system calls, character handling, and control structures. The implementation showcases practical applications of low-level programming in multimedia and user input processing.

Future Improvements:
• Dynamic Instrument Selection: Allow users to switch instruments (e.g., piano, guitar) via key commands.
• Volume Control: Add functionality to increase or decrease volume dynamically.
• Octave Expansion: Extend the key mapping to cover additional octaves.
• Real-Time Recording: Implement a feature to record and replay custom user-generated sequences.
• Visual Feedback: Display the name of the note or instrument currently being played on screen. These enhancements would expand the program's usability and bring it closer to a functional digital synthesizer prototype.

8.Bibliography:

MIDINoteFrequencyChart: https://pages.mtu.edu/~suits/notefreqs.html
RISC-V Instruction Set Manual: https://riscv.org/technical/specifications/